TecnoSalta

Tienda de componentes de PC en Java

Informe del proyecto: diseño, planificación y desarrollo por etapas

Materia: Lenguajes II · Cuarto semestre

Integrantes: Corregidor Martin Guillermo , Flores Correjidor José M, Fuentes Facundo, Tapia Lucas Fabricio , Camacho Daniel

Docente: Lucero Nancy del Carmen, Macaione Rosa Eufracia, Gonzales Jesús Matias Ramiro

Fecha de entrega: [12/11/2026]

1. Introducción

TecnoSalta es una tienda de comercio electrónico de componentes de PC. Es una aplicación de escritorio hecha por completo en Java: la interfaz está construida con JavaFX, la lógica está escrita en Java puro y los datos se guardan en un solo archivo, sin instalar nada adicional.

Este informe explica cómo se llegó al programa terminado: qué se planificó, cómo se diseñó y qué se construyó en cada una de las siete etapas.

1.1 Qué hace la tienda

La aplicación tiene dos roles, y cada uno entra a pantallas distintas después de iniciar sesión.

- **Cliente:** se registra e inicia sesión, recorre el catálogo con búsqueda por nombre y filtros por categoría y precio, ve el detalle de cada producto, arma el carrito, confirma la compra y consulta el historial de sus pedidos.
- **Administrador:** da de alta, modifica y da de baja productos (con foto y stock), gestiona las categorías y ve todos los pedidos para cambiarles el estado.

La compra es simulada: descuenta el stock y genera el pedido, pero no hay pagos reales, envíos ni correos.

1.2 Objetivos

Objetivo general. Desarrollar una tienda de comercio electrónico de escritorio, completa y funcional, hecha en Java, que aplique programación orientada a objetos y una arquitectura en capas.

Objetivos específicos:

- **Modelar** el dominio de la tienda con clases, enumeraciones y relaciones, documentadas en un diagrama de clases UML.
- **Separar** la aplicación en capas (vista, controlador, servicio y repositorio) con una sola responsabilidad cada una.
- **Construir** una interfaz gráfica con JavaFX, con cada pantalla en un archivo FXML y los estilos en CSS.
- **Guardar** los datos entre una ejecución y la siguiente sin instalar ningún programa adicional.
- **Validar** las reglas del negocio: stock disponible, precios, datos de registro y contraseñas.
- **Comprobar** la lógica del carrito y de la compra con pruebas automáticas JUnit.
- **Planificar** el trabajo en etapas con un criterio de finalización verificable para cada una.

1.3 Resumen de lo implementado

Tabla 1. Dimensión del proyecto terminado

Elemento	Cantidad	Detalle
Pantallas	8	Login, registro, catálogo, detalle, carrito, confirmación, mis pedidos y panel de administrador. Son 9 archivos FXML contando la barra superior.
Clases del modelo	9	Siete clases comunes y dos enumeraciones.

Elemento	Cantidad	Detalle
Archivos Java	37	Unas 2.350 líneas, repartidas en cinco paquetes más App y module-info.
Pruebas JUnit	12	Cinco sobre el carrito y siete sobre la compra y los pedidos.
Datos de ejemplo	17 productos	Siete categorías de componentes y dos usuarios de prueba.

2. Planificación

2.1 Forma de trabajo

El proyecto se planificó antes de escribir código..

Cada etapa tiene un criterio de “listo” que se puede comprobar ejecutando el programa. La regla fue no pasar a la etapa siguiente hasta cumplirlo, para no construir sobre algo roto. El orden va de abajo hacia arriba: primero el modelo y los datos, después los servicios y por último las pantallas.

2.2 Decisiones

Tabla 2. Decisiones tomadas en el documento de diseño

Decisión	Elección	Motivo
Lenguaje	Java 25 con JavaFX 25	Versión con soporte largo. Cada versión de JavaFX pide un JDK mínimo, por eso las dos van en la misma versión.
Interfaz	JavaFX con FXML y CSS	Cada pantalla queda en un archivo FXML y los estilos en un CSS aparte.
Datos	Listas en memoria guardadas en tienda.dat	Es simple, no requiere instalar nada y los datos siguen ahí al volver a abrir el programa.
Construcción	Maven con javafx-maven-plugin	Descarga las librerías, compila, corre las pruebas y ejecuta la aplicación.
Contraseñas	Hash con BCrypt (jBCrypt)	Nunca se guardan en texto plano.
Tema visual	AtlantaFX	Tema moderno con modo claro y modo oscuro.

2.3 Librerías y herramientas utilizadas

Se eligieron pocas a propósito, para que el código siga siendo Java claro y fácil de explicar.

Tabla 3. Librerías y herramientas del proyecto

Librería o herramienta	Versión	Para qué se usa
JavaFX Controls (org.openjfx)	25.0.1	Componentes de la interfaz: botones, tablas, campos de texto, listas y pestañas.
JavaFX FXML (org.openjfx)	25.0.1	Carga cada pantalla desde su archivo FXML y conecta los controles con su controlador.
AtlantaFX (io.github.mkpaz)	2.0.1	Tema visual moderno con modo claro y modo oscuro.
jBCrypt (org.mindrot)	0.4	Calcula y verifica el hash de las contraseñas.
JUnit Jupiter (org.junit)	5.11.4	Pruebas automáticas del carrito y de la compra.

Librería o herramienta	Versión	Para qué se usa
javafx-maven-plugin	0.0.8	Ejecuta la aplicación con mvnw.cmd javafx:run.
maven-compiler-plugin y maven-surefire-plugin	3.14.0 y 3.5.2	Compilan el código para Java 25 y corren las pruebas.
Maven Wrapper	Maven 3.9.11	Permite compilar y ejecutar sin tener Maven instalado.
jlink y jpackage	JDK 25	Arman la carpeta ejecutable con su propio Java.

De la biblioteca estándar de Java se usan, entre otras, ObjectOutputStream y ObjectInputStream para guardar y leer tienda.dat, BigDecimal para los precios, LocalDateTime para la fecha de los pedidos y las colecciones ArrayList y List para las listas en memoria.

2.4 Plan de etapas

Tabla 4. Las siete etapas y su criterio de finalización

Etapa	Qué se construye	Listo cuando
1. Proyecto base	Proyecto Maven con JavaFX y module-info.	Se abre una ventana vacía.
2. Modelo y datos	Las nueve clases del modelo, Almacen y los repositorios.	Los datos de ejemplo se leen desde tienda.dat.
3. Login, registro y navegación	AuthService, Sesion y Navegador, con sus pantallas.	Cada rol entra a su pantalla.
4. Catálogo	Tarjetas de producto, búsqueda y filtros.	Los filtros responden al escribir.
5. Carrito, compra e historial	CarritoService, PedidoService y tres pantallas.	Una compra descuenta el stock y aparece en Mis pedidos.
6. Panel de administrador	Pestañas de productos, categorías y pedidos.	Se crea un producto y se cambia el estado de un pedido.
7. Pulido y entrega	Tema, modo oscuro, pruebas, README y ejecutable.	Pasan las pruebas y el README explica cómo ejecutar.

2.5 Reglas del proyecto

- Las capas se respetan: vista, controlador, servicio, repositorio y archivo de datos.
- Los controladores nunca tocan las listas ni el archivo; eso vive solo en los repositorios y en Almacen.
- Nivel básico-intermedio: se prefiere la solución más clara, sin librerías ni patrones extra.
- El diagrama de clases del documento de diseño se mantiene igual al código.

3. Diseño

3.1 Arquitectura en capas

La aplicación se divide en cuatro capas apiladas más el modelo, que usan todas. La regla es una sola: cada capa llama únicamente a la de abajo.

Tabla 5. Capas de la aplicación

Capa	Responsabilidad	Dónde está
Vista	Define qué se ve en cada pantalla.	Archivos FXML y tienda.css
Controladores	Atienden los clics y muestran los datos. No guardan nada por su cuenta.	Paquete controlador
Servicios	Reglas del negocio: validar, filtrar, comprar.	Paquete servicio
Repositorios	Único lugar que busca y guarda en las listas.	Paquete repositorio
Archivo	Guarda los datos entre una ejecución y la siguiente.	tienda.dat
Modelo	Las entidades de la tienda. Lo usan todas las capas.	Paquete modelo

Un ejemplo de recorrido: el clic en Confirmar compra llega a ConfirmacionController, que llama a PedidoService.confirmarCompra(), que usa PedidoRepositorio para guardar en tienda.dat. Gracias a esta separación, cada parte se puede entender, cambiar y probar por separado.

Dos clases de apoyo completan la arquitectura. Sesion es un singleton, es decir, una única instancia para todo el programa, que guarda el usuario que inició sesión y su carrito. Navegador cambia de pantalla cargando el FXML que corresponde, para que ningún controlador repita ese código.

3.2 Modelo de dominio: diagrama de clases UML

Nueve clases cubren toda la tienda: siete clases comunes y dos enumeraciones.

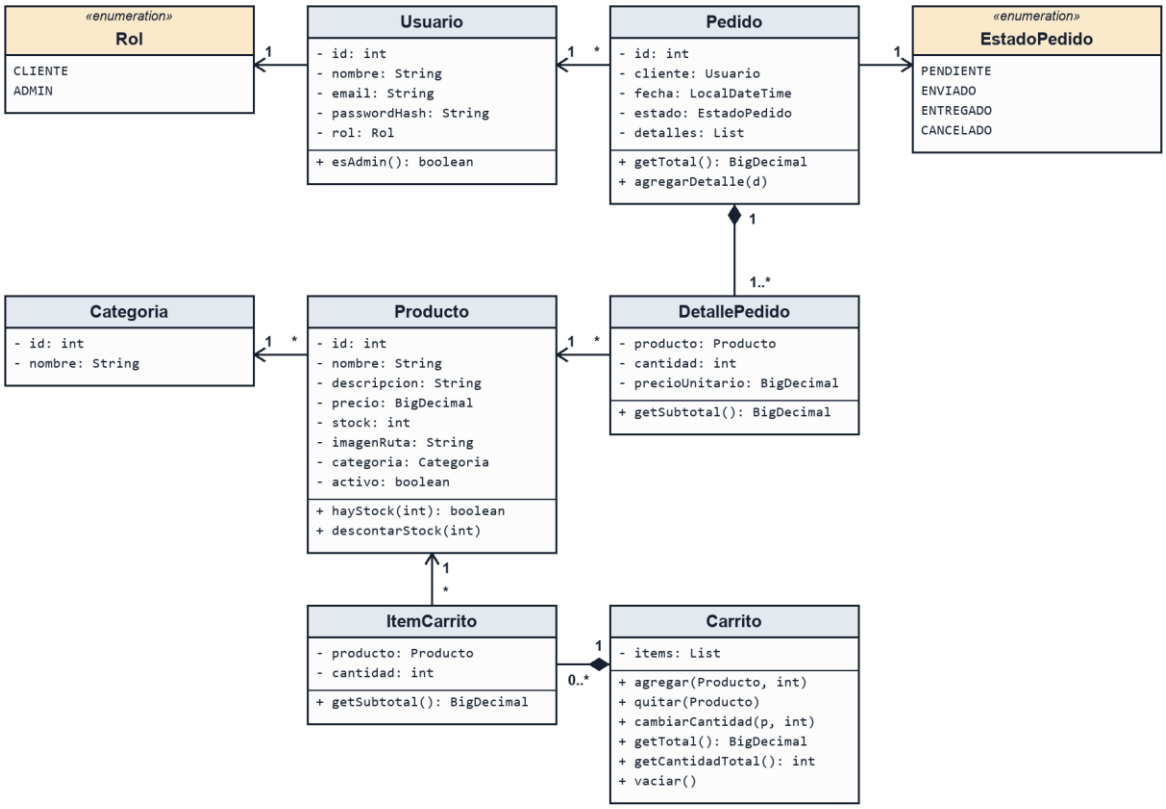


Figura 1. Diagrama de clases del modelo

Cómo se lee cada relación:

- **Pedido y Usuario.** Un cliente puede tener muchos pedidos y cada pedido es de un solo cliente. La flecha va de Pedido a Usuario porque Pedido tiene el atributo cliente.
- **Pedido y DetallePedido.** Es una composición: un renglón no existe sin su pedido, y todo pedido tiene al menos un renglón.
- **Carrito e ItemCarrito.** También es una composición, pero el carrito puede estar vacío.
- **DetallePedido e ItemCarrito con Producto.** Muchos renglones pueden apuntar al mismo producto.
- **Producto y Categoria.** Cada producto pertenece a una sola categoría.
- **Rol y EstadoPedido.** Son enumeraciones: tienen valores fijos que controla el compilador, en lugar de textos donde un error de tipeo pasa desapercibido.

ItemCarrito y DetallePedido se parecen, pero son clases distintas a propósito. ItemCarrito es temporal y siempre usa el precio actual del producto. DetallePedido es permanente y guarda precioUnitario, el precio del momento de la compra.

3.3 Servicios

Tabla 6. Clases del paquete servicio

Clase	Qué resuelve
AuthService	Registro e inicio de sesión. Valida los datos, guarda la contraseña como hash BCrypt y deja al usuario en la Sesion.
ProductoService	Lista y filtra el catálogo, valida y guarda productos, hace el borrado lógico y gestiona las categorías.
CarritoService	Agrega, quita y cambia cantidades en el carrito de la sesión, validando siempre el stock antes.
PedidoService	Confirma la compra, lista los pedidos de un cliente o de todos y cambia el estado. Al cancelar devuelve el stock.
Sesion	Singleton con el usuario que inició sesión y su carrito.
StockInsuficienteException	Excepción propia que se lanza si se pide más cantidad que el stock disponible.

3.4 Datos y persistencia

Mientras el programa corre, todo vive en listas de Java dentro de la clase Almacen. Cada vez que cambia algo, Almacen se guarda entero en el archivo tienda.dat con serialización (ObjectOutputStream), y al arrancar se vuelve a leer (ObjectInputStream). El carrito no se guarda: vive en memoria mientras dura la sesión.

Como se escribe el objeto completo de una vez, las referencias entre objetos se conservan: el producto de un pedido sigue siendo el mismo objeto que está en la lista de productos.

- **Borrado lógico.** Un producto que ya aparece en pedidos no se borra: se marca como inactivo y deja de mostrarse en el catálogo.
- **Precio congelado.** DetallePedido guarda el precio unitario, así un pedido viejo no cambia si después se modifica el precio del producto.
- **Serializable.** Las clases del modelo que se guardan implementan Serializable. Si cambian sus atributos, hay que borrar tienda.dat para que el programa lo cree de nuevo.

- **Ids y dinero.** Almacen lleva un contador por lista para asignar el id de cada objeto nuevo. Los precios se manejan con BigDecimal, redondeados a dos decimales.

Si tienda.dat no existe, el programa carga los datos de ejemplo desde la clase DatosIniciales.

3.5 Pantallas y navegación

Desde el login se pasa al registro, al catálogo (si entra un cliente) o al panel de administrador. El cliente tiene siempre a mano una barra superior con Catálogo, Carrito (con contador), Mis pedidos, el cambio de tema y Cerrar sesión.

Tabla 7. Pantallas de la aplicación

Pantalla	Componentes JavaFX principales
Login	TextField, PasswordField, Button y un Hyperlink al registro.
Registro	Campos con validación de email y contraseña.
Catálogo	Barra superior, campo de búsqueda, ComboBox de categorías, filtro de precio y FlowPane de tarjetas dentro de un ScrollPane.
Detalle	Ventana modal con la imagen, la descripción, el precio y un Spinner de cantidad.
Carrito	TableView con un Spinner por renglón, total y botón Continuar.
Confirmación	Resumen del pedido, total y botones Confirmar y Volver.
Mis pedidos	TableView de pedidos y, al seleccionar uno, sus renglones.
Panel de administrador	TabPane con tablas y formularios por pestaña, y FileChooser para la imagen.

4. Desarrollo por etapas

Etapa 1. Proyecto base

Objetivo. Tener un proyecto que compile y abra una ventana.

Qué se hizo. Se creó el proyecto Maven con el archivo pom.xml, que declara las librerías (JavaFX, AtlantaFX, jBCrypt y JUnit) y el plugin que ejecuta la aplicación. Se escribió module-info.java, que indica qué librerías usa el módulo y le abre el paquete de controladores a JavaFX, porque FXML crea los controladores por reflexión. La clase App arranca JavaFX.

Etapa 2. Modelo y datos

Objetivo. Tener las entidades de la tienda y un lugar donde guardarlas.

Qué se hizo. Se escribieron las nueve clases del modelo según el diagrama, con atributos privados, constructores, getters y setters. Después se hizo Almacen, con las cuatro listas y los métodos para guardar y cargar tienda.dat, y un repositorio por entidad (usuarios, categorías, productos y pedidos). DatosIniciales carga los usuarios de prueba y los productos de ejemplo.

Resultado. La primera ejecución crea tienda.dat con los datos de ejemplo y la segunda los lee desde el archivo.

Etapa 3. Login, registro y navegación

Objetivo. Que cada usuario entre a la pantalla que le corresponde.

Qué se hizo. AuthService registra clientes e inicia sesión. Valida el nombre, el formato del email y que la contraseña tenga al menos seis caracteres, y rechaza un email repetido. La contraseña se guarda como hash BCrypt. Si el email no existe o la contraseña no coincide, el mensaje es el mismo en los dos casos, para no revelar qué emails están registrados. Sesion recuerda al usuario y Navegador abre el catálogo o el panel de administrador según el rol.

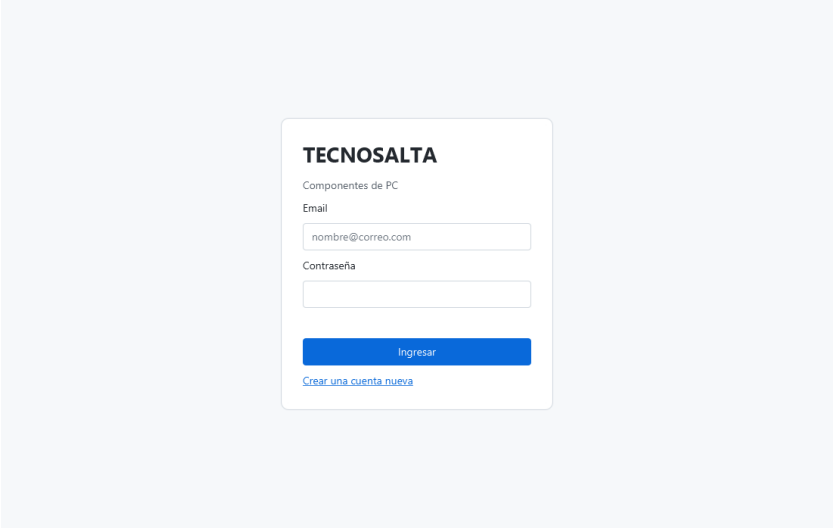


Figura 2. Pantalla de inicio de sesión

Etapa 4. Catálogo

Objetivo. Mostrar los productos y permitir encontrarlos.

Qué se hizo. Cada producto se dibuja como una tarjeta con su nombre, categoría, precio y botón Agregar. Arriba hay tres filtros que se combinan: texto en el nombre, categoría y precio máximo. Los filtros responden mientras se escribe, porque el controlador escucha los cambios de cada campo y vuelve a pedir la lista. La decisión de qué productos coinciden la toma ProductoService.buscar(), no el controlador:

```
for (Producto producto : listarActivos()) {
    boolean coincideNombre = producto.getNombre().toLowerCase().contains(buscado);
    boolean coincideCategoria = categoria == null
            || producto.getCategoria().getId() == categoria.getId();
    boolean coincidePrecio = precioMaximo == null
            || producto.getPrecio().compareTo(precioMaximo) <= 0;
    if (coincideNombre && coincideCategoria && coincidePrecio) {
        resultado.add(producto);
    }
}
```

Cuando un producto no tiene unidades, el botón se deshabilita y dice “Sin stock”. Un clic en la tarjeta abre el detalle, donde se elige la cantidad.

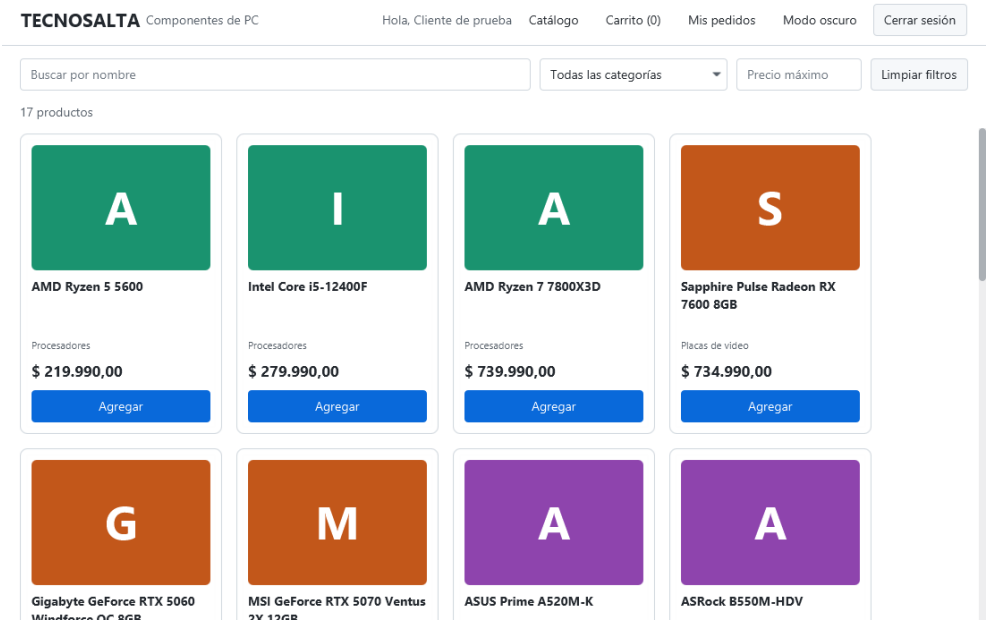


Figura 3. Catálogo con los componentes de ejemplo

Etapa 5. Carrito, compra e historial

Objetivo. Completar el recorrido de compra del cliente.

Qué se hizo. CarritoService valida el stock antes de agregar un producto o cambiar una cantidad, contando también las unidades que ya estaban en el carrito. Si no alcanza, lanza StockInsuficienteException. En la pantalla del carrito cada renglón tiene un Spinner que va de 1 al stock del producto.

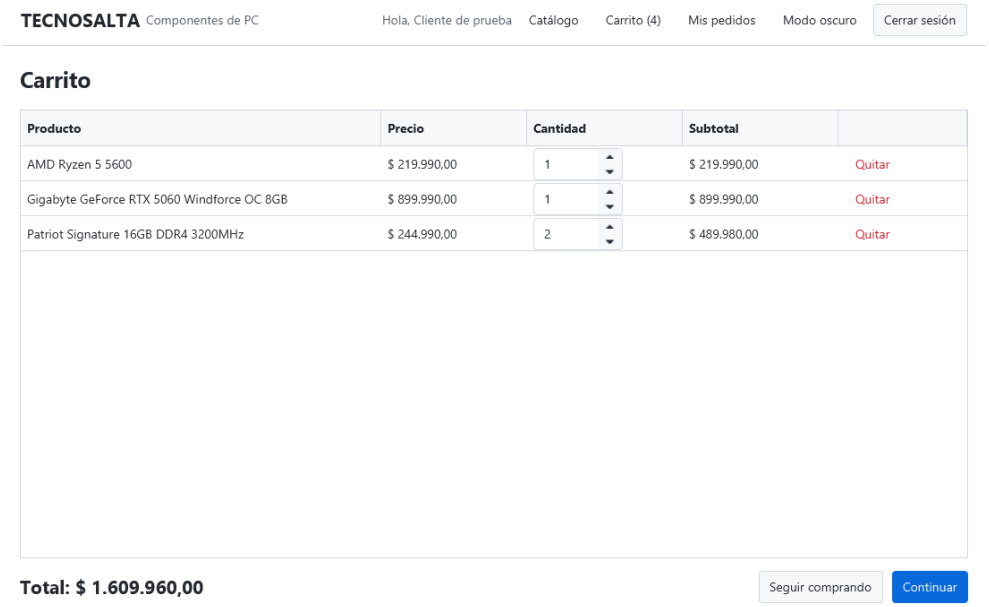


Figura 4. Carrito con un selector de cantidad por renglón

La compra se resuelve en PedidoService.confirmarCompra() y es “todo o nada”. Son tres pasos y el orden importa:

```
// 1) Verificar todo antes de tocar nada
for (ItemCarrito item : carrito.getItems()) {
    if (!item.getProducto().hayStock(item.getCantidad())) {
        throw new StockInsuficienteException(item.getProducto(), item.getCantidad());
    }
}
// 2) Crear el pedido con el precio congelado y descontar el stock
Pedido pedido = new Pedido(sesion.getUsuario());
for (ItemCarrito item : carrito.getItems()) {
    Producto producto = item.getProducto();
    pedido.agregarDetalle(new DetallePedido(producto, item.getCantidad(),
producto.getPrecio()));
    producto.descontarStock(item.getCantidad());
}
// 3) Guardar una sola vez y vaciar el carrito
pedidos.guardar(pedido);
carrito.vaciar();
```

Si falta stock de un solo producto, la excepción sale antes de modificar algo y no se guarda nada.

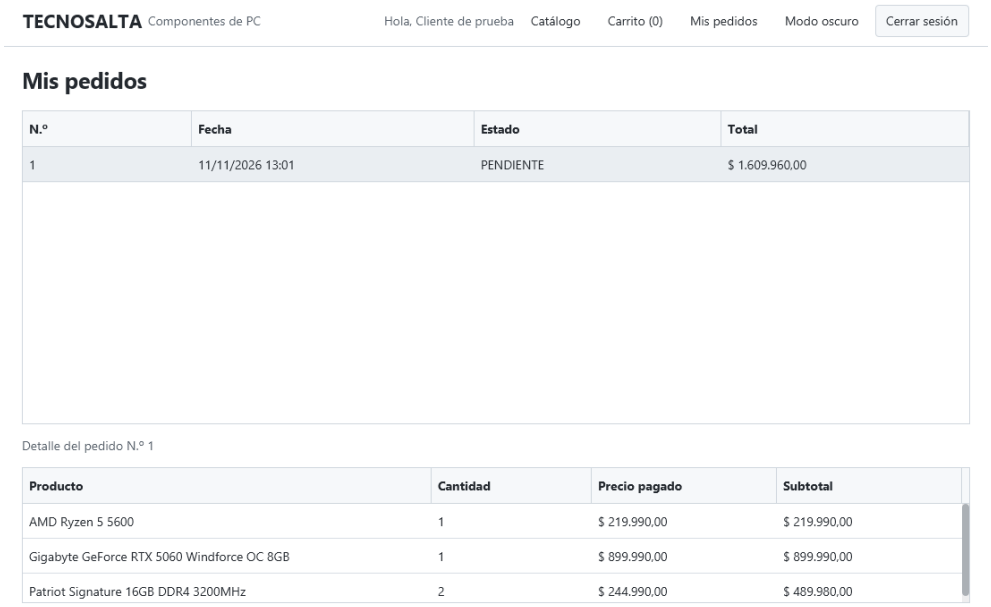


Figura 5. Historial de pedidos con el detalle del pedido seleccionado

Etapa 6. Panel de administrador

Objetivo. Que el administrador pueda mantener la tienda sin tocar el código.

Qué se hizo. El panel es un TabPane con tres pestañas. En Productos, al seleccionar un renglón sus datos pasan a un formulario; ProductoService valida el nombre, el precio, el stock y la categoría antes de guardar. La foto se elige con un FileChooser y se copia a la carpeta imagenes. Dar de baja es un borrado lógico. En Categorías no se puede repetir un nombre ni eliminar una categoría que tiene productos. En Pedidos se ve el detalle y se cambia el estado; cancelar un pedido devuelve el stock y un pedido cancelado no se puede reabrir.

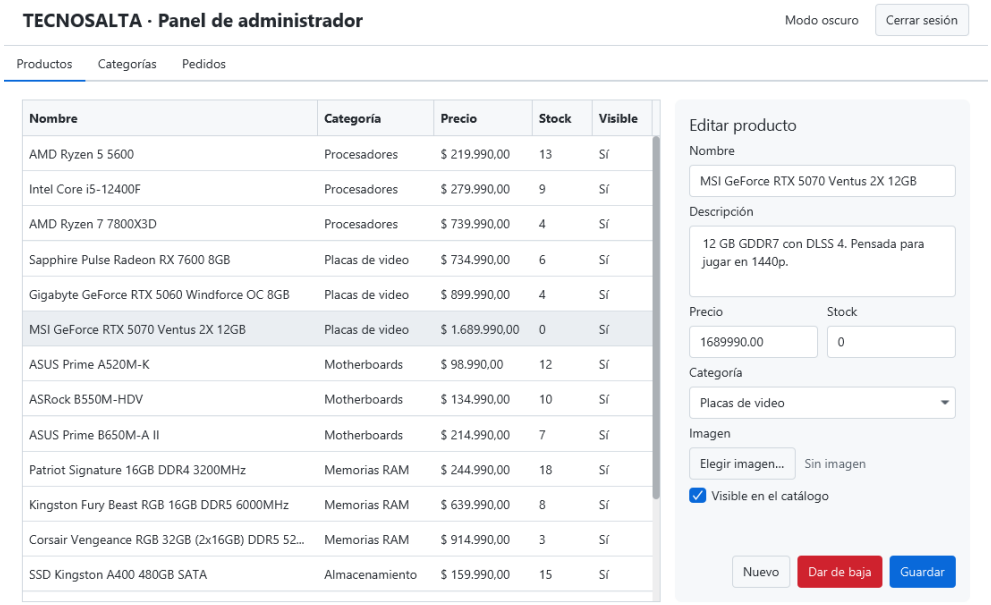


Figura 6. Pestaña Productos del panel de administrador

Etapa 7. Pulido y entrega

Objetivo. Dejar el proyecto presentable

Qué se hizo. Se aplicó el tema AtlantaFX con un botón para alternar entre modo claro y modo oscuro, y un archivo tienda.css con los estilos propios. Se escribieron las doce pruebas Junit..

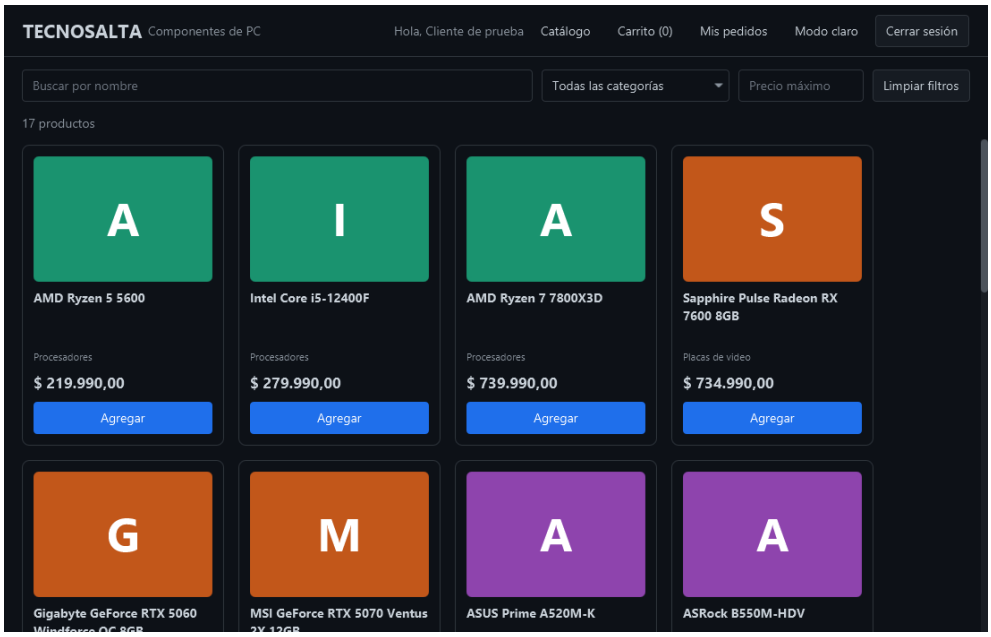


Figura 7. El catálogo en modo oscuro

5. Cambios durante el desarrollo

El proyecto tuvo dos cambios de rumbo. La arquitectura en capas permitió absorberlos sin rehacer el resto.

- **Ejecutable independiente.** Se agregó la carpeta ejecutable para poder mostrar el programa en cualquier computadora con Windows sin instalar nada.
- **TecnoSalta y precios reales.** La tienda genérica pasó a llamarse TecnoSalta y a vender componentes de PC. Los 17 productos de ejemplo tienen precios en pesos argentinos tomados como referencia de tiendas de hardware del país en noviembre de 2026. El paquete Java sigue siendo com.tienda.

6. Pruebas y verificación

Las pruebas automáticas apuntan a la lógica más delicada: el carrito y la compra. Cada prueba arma sus propios datos en memoria, así que no lee ni modifica tienda.dat

Tabla 8. Las doce pruebas JUnit

Clase de prueba	Qué comprueba
CarritoServiceTest	Agregar el mismo producto suma la cantidad.
CarritoServiceTest	El total suma los subtotales.
CarritoServiceTest	No se puede agregar más que el stock.
CarritoServiceTest	Cambiar la cantidad valida el stock.
CarritoServiceTest	Quitar saca el producto del carrito.
PedidoServiceTest	Confirmar la compra crea el pedido, descuenta el stock y vacía el carrito.
PedidoServiceTest	Si falta stock de un producto, no se guarda nada.
PedidoServiceTest	Con el carrito vacío no se puede comprar.
PedidoServiceTest	El pedido guarda el precio del momento de la compra.
PedidoServiceTest	Cada cliente ve solo sus pedidos.
PedidoServiceTest	Cancelar un pedido devuelve el stock.
PedidoServiceTest	Un pedido cancelado no se puede reabrir.

Además de las pruebas JUnit, durante el desarrollo se recorrieron las pantallas de forma automática: login con contraseña incorrecta, registro, filtros del catálogo, agregar al carrito, cambiar una cantidad, quitar un producto, alta y edición de un producto, cancelación de un pedido y alta de una categoría. También se comprobó que la aplicación arranca y que la segunda vez lee tienda.dat.7. Cómo ejecutar el programa

7 ETAPA

- **Desde el ejecutable:** abrir la carpeta ejecutable\TecnoSalta y hacer doble clic en TecnoSalta.exe. No hace falta tener Java instalado.
- **Usuarios de prueba:** hay un administrador y un cliente que se crean solos en el primer arranque.
  Admin :admin@tecnosalta contraseña admin123
  Cliente : cliente@tecnosalta contraseña cliente123

- **Volver a los datos de ejemplo:** borrar el archivo tienda.dat y abrir el programa de nuevo.

8. Estado de la entrega

Tabla 9. Lista de entrega del documento de diseño

Punto de la lista de entrega	Estado
El diagrama de clases UML coincide con el código final	Cumplido
Usuarios de prueba anotados en el README	Cumplido
README con requisitos y forma de ejecutar	Cumplido
Capturas de pantalla de catálogo, carrito y panel de administrador	Cumplido
Pruebas JUnit pasando con mvnw.cmd test	Cumplido (12 de 12)
Repositorio Git con un commit por etapa	Pendiente: la carpeta todavía no es un repositorio

9. Conclusiones

El proyecto cumple el objetivo general: una tienda completa para dos roles, hecha solo en Java, que se puede abrir y mostrar sin preparar nada. Los objetivos específicos también se cumplieron: el dominio está modelado y documentado en UML, la aplicación está separada en capas, la interfaz es JavaFX con FXML y CSS, los datos se guardan sin servidor, las reglas del negocio se validan en los servicios y la lógica de la compra está cubierta por pruebas automáticas.

Lo que más pesó en el resultado fue haber diseñado antes de programar. Tener las capas definidas desde el principio permitió cambios de rumbo grandes, como convertir la tienda en una de componentes de PC, tocando pocas clases y sin rehacer las pantallas.

Los puntos que distinguen al trabajo son de ingeniería más que de cantidad de pantallas: la compra todo o nada, el precio congelado en el pedido, las contraseñas con BCrypt, una excepción propia para un error del negocio, el respeto estricto de las capas y las pruebas automáticas.

Como aprendizaje queda la importancia de fijar un criterio de “listo” para cada etapa: obligó a tener siempre una versión que funcionara y evitó acumular errores para el final.

10. Cómo seguir y profundizar el proyecto

El proyecto quedó preparado para crecer. Las propuestas van de lo más inmediato a lo que requiere estudiar temas nuevos.

10.1 Próximos pasos inmediatos

- **Repositorio Git.** Es el único punto pendiente de la lista de entrega. Conviene crearlo y registrar los cambios con un commit por etapa.

- **Fotos reales.** Cargar desde el panel de administrador una foto para cada producto, que hoy muestran un recuadro de color con la inicial.
- **Más pruebas.** Agregar pruebas para AuthService (email repetido, contraseña corta, inicio de sesión fallido) y para ProductoService (filtros y validaciones).
- **Pruebas de la interfaz.** Usar TestFX para automatizar recorridos por las pantallas, como el login y la compra.

10.2 Nuevas funciones

- **Reportes para el administrador.** Ventas por día, productos más vendidos y productos con poco stock.
- **Mejoras del catálogo.** Ordenar por precio o por nombre, paginar los resultados y filtrar por precio mínimo.
- **Exportar pedidos.** Generar un comprobante en PDF o un archivo CSV con los pedidos.
- **Instalador.** Usar jpackage para generar un instalador .msi en lugar de una carpeta.

10.3 Temas para profundizar en Lenguajes II

- **Genéricos.** Los cuatro repositorios repiten la lógica de asignar el id y guardar. Un Repositorio<T> genérico escribiría esa lógica una sola vez.
- **Streams y lambdas.** Reescribir los filtros de ProductoService y los cálculos de totales con la API de streams, y comparar la legibilidad con la versión actual.
- **Hilos.** Guardar tienda.dat en un hilo aparte (por ejemplo con Task de JavaFX) para que la ventana no se congele cuando haya muchos datos.
- **Interfaces.** Definir una interfaz para cada repositorio, para que el resto del programa no dependa de cómo se guardan los datos.
- **Versión web.** Como los servicios no dependen de JavaFX, se podrían reutilizar detrás de una API web hecha, por ejemplo, con Spring Boot.